

Communicating by Value: Budget-Aware Federated Stream Clustering over Apache Kafka for Evolving Non-IID Edge Streams

Author One, *Student Member, IEEE*, Author Two, *Student Member, IEEE*, and Author Three, *Member, IEEE*

Abstract—Edge devices produce unlabeled, non-IID data streams over constrained links, and operators need a continuously updated global clustering without collecting raw data. Every device must repeatedly decide whether an update is worth its network cost and which part of its local model to send. Existing systems answer with a schedule (periodic or one-shot federated clustering) or, as in communication-efficient federated learning, by the magnitude of the change. We show that the right quantity is the *value* of an update. When a coordinator computes a cluster centre from stale summaries, its k-means cost exceeds the optimum by exactly $\|\Delta L_j - \Delta N_j \hat{c}_j\|^2 / N_j$, a closed form that every device can evaluate for its own micro-clusters from the broadcast model. Growth of a cluster in place is worth nothing, whereas the first points of an emerging cluster are worth a lot. We build FedCAST on this criterion: value-greedy selection of cluster-feature deltas, a scale-free dual-ascent threshold with a hard token-bucket budget and use-it-or-lose-it sends, 8-bit deltas with error feedback, and a staleness-robust approximation guarantee $J \leq \gamma \text{OPT} + (1 + \gamma)U$. FedCAST runs on Apache Kafka, whose per-key ordering, idempotent producers and log-compacted changelog provide exactly-once state and crash recovery, in under 1 GB of RAM. Over 3,800 seeded runs on eight datasets, including three naturally drifting real streams, value-based communication beats the strongest federated-learning compressor (top- k magnitude with error feedback, equally quantized) in 72 % of 200 paired non-stationary comparisons ($p = 4 \times 10^{-11}$) and in 75 % on naturally drifting real data ($p = 2 \times 10^{-6}$), while tying on static streams. Both far outperform periodic synchronization and k-FED. A multi-container deployment with kernel-enforced bandwidth caps confirms the gains end to end.

Index Terms—Federated clustering, data streams, concept drift, communication efficiency, non-IID data, edge computing, Apache Kafka.

I. INTRODUCTION

NETWORK probes, industrial gateways, sensor motes and vehicles continuously produce unlabeled data streams at the edge. Operators want one continuously updated *global* view of the structure in these streams, such as the traffic profiles present in a network right now or the operating regimes of a sensor field. Three constraints stand in the way. Raw data often cannot leave the device, for legal, privacy or volume reasons. The data are *non-IID*: each device sees a different slice of the global distribution, and some patterns appear at a single device only. And links are constrained: metered cellular uplinks, lossy wireless and intermittent connectivity make every byte costly.

The authors are with the Department of Computer Science, University Name, City, Country (e-mail: author@university.edu).

Code, configurations, raw results and scripts that regenerate every figure and table are publicly available in the project repository.

Stream clustering handles the first constraint with compact, additive micro-cluster summaries [1]–[3]. Distributed variants ship these summaries to a coordinator [4]–[6], and federated clustering copes with heterogeneity on static data [7], [8]. Every edge node, however, faces a question at every instant that none of these works answers in a principled way: *is sending an update now worth its network cost, and if so, which part of the model should be sent?* Schedules (periodic updates, one-shot federated clustering) waste bytes when nothing changes and leave the coordinator stale exactly when the data drift. Communication-efficient federated learning ranks updates by their *magnitude*, e.g. top- k sparsification with error feedback [9]–[11]. As we show, magnitude is the wrong currency for clustering: large changes are often worthless (a cluster growing in place), and small ones can be critical (the first points of a new cluster).

Key insight. With the assignment of summaries to centres fixed, the extra k-means cost that staleness causes at the coordinator has an *exact closed form*: for a centre $\hat{c}_j$ computed from stale summaries, the cost exceeds its optimum by $\|\Delta L_j - \Delta N_j \hat{c}_j\|^2 / N_j$, where $\Delta L_j, \Delta N_j$ are the unsent changes of the linear sum and mass of cluster j (Proposition 1). Each device knows its own contribution to these quantities and receives $\hat{c}_j, N_j$ with the global model, so it can compute the exact *value* of sending each of its micro-clusters.

Contributions.

- 1) *Value of an update.* The closed-form value above, its marginal form for greedy selection, and the observation that it is zero for in-place growth and large for emerging mass.
- 2) *FedCAST*, a budget-aware federated stream clustering system built on this value: value-greedy cluster-feature (CF) deltas with a magnitude tie-break, a scale-free exponentiated dual-ascent threshold, a hard token-bucket budget with use-it-or-lose-it sends, and 8-bit deltas with error feedback.
- 3) *Guarantees*: a staleness-robust approximation bound $J(S; \hat{C}) \leq \gamma \text{OPT}(S) + (1 + \gamma)U$ with a model-free, locally computable staleness U (Theorem 1), and a hard and long-run budget guarantee (Proposition 2).
- 4) *A Kafka-native system*: per-key ordering, idempotent producers with sequence numbers, and a log-compacted changelog give exactly-once state and crash recovery without resends, in under 1 GB of RAM.
- 5) *A rigorous evaluation*: 3,800+ seeded runs on eight datasets (static, drifting, evolving, and three *naturally*

drifting real streams, one of them naturally partitioned by device), paired statistical tests against an equally quantized top- k compressor, real-broker fault injection, and a multi-container deployment with kernel bandwidth caps. We also report the ideas that did *not* work.

II. RELATED WORK

Stream clustering. CluStream [2] introduced online micro-clusters with offline macro-clustering, building on BIRCH cluster features [1]. DenStream [3] added decay and density-based macro-clustering, and DBSTREAM [12] a shared-density graph; see [13], [14] for surveys of stream clustering and concept drift.

Distributed stream clustering. Cormode et al. [4] maintain a continuous k -center clustering over distributed streams. DGClust [5] reports grid-cell state changes, and Tran and Sattler [6] ship micro-clusters to a coordinator. Coresets [15] give strong guarantees on static data. These methods send on fixed rules and have neither a byte budget nor a notion of the value of an update.

Federated clustering. k -FED [7] performs one-shot federated k -means and shows that heterogeneity can help. FFCM [16] and AFCL [8] extend federated clustering to fuzzy memberships and to asynchrony with unknown k . All assume static local data. FedCluLearn [17] and federated anomaly detection on streams [18], [19] target supervised or anomaly-detection objectives.

Communication-efficient learning. Federated learning [20], [21] reduces traffic by sparsification (top- k) [9], [22], [23], quantization [24] and error feedback [10], [11], and by event-triggered communication [25], [26]. These methods rank coordinates by magnitude. We use sparsification, quantization and error feedback as building blocks, but rank by the exact effect on the clustering objective, and we compare against magnitude ranking under identical quantization.

Kafka for machine learning. Kafka-ML [27], [28] uses Kafka to train neural networks federatedly. We use Kafka's semantics as part of the correctness and recovery argument, following the changelog pattern of stream processors [29], [30].

III. PROBLEM FORMULATION

There are m edge nodes. Node i receives points $x \in \mathbb{R}^d$ from a time-varying, non-IID distribution $P_i(t)$, and the global distribution is the mixture $P(t)$. Node i summarizes its stream by at most q micro-clusters with time-decayed cluster features [1], [3]

$$\text{CF} = (n, \mathbf{L}, SS, \tau),$$

$$n = \sum w_x, \quad \mathbf{L} = \sum w_x x, \quad SS = \sum w_x \|x\|^2, \quad (1)$$

with weights $w_x = 2^{-(t-t_x)/H}$ and last update τ . Decay scales all three sums by one factor, so centroids and radii are decay-invariant, and a copy of a CF plus its τ determines its decayed value at any time. The k -means cost of the points summarized by a CF at centre c is

$$J(\text{CF}, c) = SS - 2\mathbf{L}^\top c + n\|c\|^2. \quad (2)$$

The coordinator stores the last CFs received from each node, $\hat{S}_i$, and publishes global centres $C = \{c_j\}$ with masses N_j . Node i may send at most B_i bytes per window W over a link with price $p_i \geq 1$. The goal is to keep $J(C(t); P(t))$ close to a centralized learner with all raw data, subject to the budgets.

IV. FEDCAST

A. Local Summarization

Each node runs a CluStream-style update [2] with DenStream-style decay [3]. A point joins the nearest micro-cluster if it lies within twice its RMS radius (with a floor); otherwise it opens a new one. When all q slots are full, a stale micro-cluster is pruned or the two closest are merged, using an exact nearest-neighbour cache updated in $O(qd)$. The node keeps an exact copy of the CFs it has sent, which is precisely the coordinator's view, because both sides apply the same deterministic decay.

B. The Value of an Update

Proposition 1 (Exact staleness excess). *Fix the assignment of CFs to global clusters. Let the coordinator's centre of cluster j be $\hat{c}_j = \hat{L}_j / \hat{N}_j$, computed from the stale summaries, and let the true aggregates be $L_j = \hat{L}_j + \Delta L_j$ and $N_j = \hat{N}_j + \Delta N_j$. Then*

$$J_j(\hat{c}_j) - \min_c J_j(c) = \frac{\|\Delta L_j - \Delta N_j \hat{c}_j\|^2}{N_j}. \quad (3)$$

Proof: By (2), $J_j(c) = SS_j - 2\mathbf{L}_j^\top c + N_j\|c\|^2$ is minimized at $c^* = L_j/N_j$, and $J_j(\hat{c}) - J_j(c^*) = N_j\|\hat{c} - c^*\|^2 = \|N_j\hat{c} - L_j\|^2/N_j$. Since $\hat{N}_j\hat{c}_j = \hat{L}_j$, we have $N_j\hat{c}_j - L_j = \Delta N_j\hat{c}_j - \Delta L_j$. ■

Corollary 1. *Growth of a cluster in place ($\Delta L_j = \Delta N_j \hat{c}_j$) has zero value, however large ΔN_j is. Mass ΔN arriving at distance δ from $\hat{c}_j$ has value $\Delta N^2 \delta^2 / N_j$: quadratic in distance and inversely proportional to the size of the global cluster.*

Magnitude-based compressors [9], [10] rank updates by $\|(\Delta n, \Delta \mathbf{L}, \Delta SS)\|$. They therefore spend bytes on the first kind of change and may delay the second, which is exactly what matters under concept evolution.

Marginal value. Node i assigns each dirty micro-cluster to its nearest broadcast centre and accumulates its own residual $r_j = \sum (\Delta \mathbf{L}_{id} - \Delta n_{id} \hat{c}_j)$ over all unsent changes, including deletions and micro-clusters that changed cluster. Sending micro-cluster id removes its contribution u_{id} from r_j and reduces the excess by

$$v_{id} = (2u_{id}^\top r_j - \|u_{id}\|^2) / N_j, \quad (4)$$

which is exact for the node's own contribution (other nodes' residuals are unknown to it and enter only through cross terms). Micro-clusters are ranked by v_{id} . Change magnitude, normalized and weighted by $\epsilon = 0.1$, breaks ties among zero-value refinements: *value first, information second*.

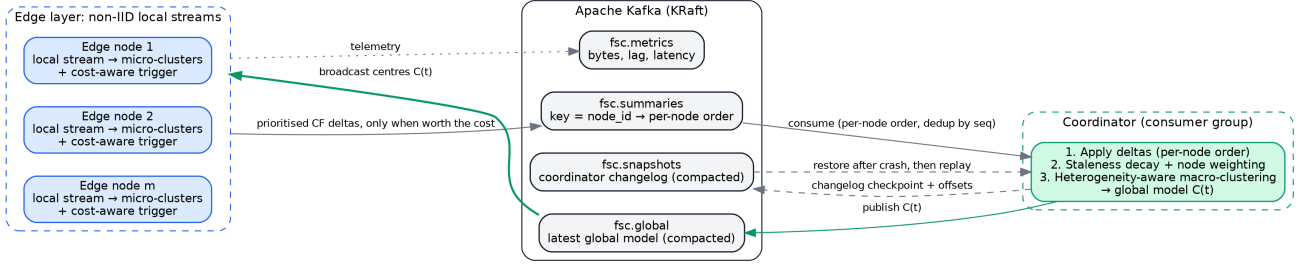


Fig. 1. FedCAST over Apache Kafka. Edge nodes publish value-ranked, 8-bit CF deltas only when they are worth their byte cost. The coordinator applies them exactly once in per-node order, checkpoints to a compacted changelog, and broadcasts the global model (centres and masses) on a compacted topic, from which the edges compute the value of their updates.

C. Budgeted Sending

Once per tick, a node selects the smallest top-ranked set of micro-clusters covering a fraction $\gamma = 0.9$ of its total score. Together with pending deletions, this set yields a payload of b bytes with covered score D . The node sends if

$$D \geq \lambda_i p_i b \quad \text{or the token bucket is about to overflow,} \quad (5)$$

provided the bucket (rate B_i/W , capacity κ_i) holds b tokens. Tokens above capacity are lost, so a send when the bucket is nearly full has no opportunity cost (*use-it-or-lose-it*). This prevents the node from under-spending once all remaining changes have near-zero value. Once per window, the threshold is updated by exponentiated dual ascent,

$$\lambda_i \leftarrow \lambda_i \exp(\eta \text{clip}_{[-1,2]}((\text{spent}_i - B_i)/B_i)), \quad (6)$$

which is scale-free: the same $\eta = 0.5$ works on every dataset. The price p_i is an exponentially weighted estimate of retransmission overhead and delivery latency, obtained from producer delivery reports.

Proposition 2 (Budget). (i) Over any interval of length T , node i sends at most $\kappa_i + B_i T/W$ bytes. (ii) Without overflow sends, if $D \leq D_{\max}$ and $b \geq b_{\min}$, the average relative overspend after n windows is at most $\log(\lambda^\dagger e^{2\eta}/\lambda_0)/(\eta n)$ with $\lambda^\dagger = D_{\max}/b_{\min}$.

Proof: (i) Every send, including an overflow send, consumes its size from a bucket that starts at κ_i , refills at rate B_i/W , and never goes negative. (ii) Let $\theta = \log \lambda$. Telescoping (6) gives $\sum_w g_w = (\theta_{n+1} - \theta_1)/\eta$. If $\lambda > \lambda^\dagger$, no send satisfies (5), so $g_w = -1$. Hence θ never exceeds $\max(\theta_1, \log \lambda^\dagger) + 2\eta$. ■

D. 8-bit Deltas with Error Feedback

A delta record carries an identifier, the count (float32), the variance (float16), a time offset (float32) and the centroid quantized to 8 bits per dimension relative to the message's own range (two float16 values per dimension). This is $14 + d$ bytes per record instead of $24 + 4d$. The node stores the *dequantized* record as its copy of the coordinator's view. Quantization error therefore shows up as ordinary staleness and is corrected by later sends (error feedback [11]), and the coordinator's state always equals the node's record of it.

Algorithm 1 FedCAST edge node i (one tick)

- 1: ingest points into the decayed CF micro-clusters
- 2: refill token bucket; if no global model yet, send a bootstrap summary
- 3: for dirty micro-clusters: residuals r_j and values v_{id} (Eq. 4)
- 4: $\text{score} \leftarrow v / \sum v + \epsilon \cdot \text{magnitude} / \sum \text{magnitude}$
- 5: $P \leftarrow$ top-scored prefix covering γ of the score; $b \leftarrow$ bytes of P (8-bit)
- 6: **if** ($D \geq \lambda p b$ **or** bucket nearly full) **and** tokens $\geq b$ **then**
- 7: produce delta to `fsc.summaries`, key `node- $i$` ; store dequantized copy
- 8: **end if**
- 9: on `fsc.global`: update $\hat{c}_j, N_j$; at window end: update λ (Eq. 6)

E. Aggregation

The coordinator applies each delta exactly once (sequence numbers per node), decays all CFs to the current time so that silent nodes fade out, and runs weighted k-means over CF centroids. It warm-starts from the previous model and adds swap local search [31] and a k-means++ restart [32]. Warm starts keep cluster identities stable, and swaps prevent a cluster that appears at a single node from being absorbed by a warm-started neighbour. The coordinator broadcasts centres, radii and masses N_j .

F. A Staleness-Robust Approximation Guarantee

Proposition 1 is exact but holds for fixed assignments. The following model-free bound holds for *any* aggregation algorithm.

Theorem 1 (Staleness-robust approximation). *Let every centre satisfy $\|c\| \leq R$, and let*

$$U = \sum_i \sum_{id} (|\Delta SS_{id}| + 2R \|\Delta \mathbf{L}_{id}\| + R^2 |\Delta n_{id}|)$$

be the model-free staleness of all nodes (current versus coordinator copies). If the coordinator computes a γ -approximate k-means solution $\hat{C}$ on the stale summaries $\hat{S}$, then on the current summaries S ,

$$J(S; \hat{C}) \leq \gamma \text{OPT}(S) + (1 + \gamma) U. \quad (7)$$

Proof: For $\|c\| \leq R$, by (2), $|J(\text{CF}, c) - J(\hat{\text{CF}}, c)| = |\Delta SS - 2\Delta \mathbf{L}^\top c + \Delta n \|c\|^2| \leq |\Delta SS| + 2R \|\Delta \mathbf{L}\| + R^2 |\Delta n|$. Since $|\min_c a_c - \min_c b_c| \leq \max_c |a_c - b_c|$, summing over

micro-clusters gives $|J(S; C) - J(\hat{S}; C)| \leq U$ for every C in the R -ball. Optimal centres are convex combinations of data points and lie in the ball. Therefore $J(S; \hat{C}) \leq J(\hat{S}; \hat{C}) + U \leq \gamma \text{OPT}(\hat{S}) + U \leq \gamma \text{OPT}(S) + U + U$. ■

With 8-bit deltas, $\hat{S}$ is the dequantized view, so U also covers quantization error. Both the exactness of (3) and the bound of Theorem 1 are checked numerically in the test suite.

Complexity. Per point: $O(qd)$. Per tick and node: $O(|\text{dirty}|kd)$. Per aggregation: $O(I m q kd)$ over at most $m q$ micro-clusters, never over raw points.

V. SYSTEM DESIGN ON APACHE KAFKA

Four Kafka semantics are part of FedCAST’s correctness argument. *Per-partition order*, with node identifier as key, lets a delta assume its predecessor. *Idempotent producers* plus per-node sequence numbers make retries on lossy links harmless, and replays after a crash are ignored. The *replayable log* with committed offsets means nodes never resend after a coordinator crash. *Log compaction* keeps one changelog record per node in `fsc.snapshots` (server-side CFs with the applied sequence number and offsets as headers) and the latest model in `fsc.global`. On restart, the coordinator restores the compacted changelog and replays `fsc.summaries` from the checkpointed offsets, the state-store pattern of stream processors [30]. Messages use a fixed binary layout whose size is known before encoding, as (5) requires. The broker runs in KRaft mode with a 256 MB heap. The same node, policy, codec and coordinator classes run both against a real broker and inside a deterministic discrete-event simulator. The simulator emulates per-node links (delay, jitter, bandwidth, loss with counted retransmissions, outages) while preserving per-key order, and its byte counts were validated against Kafka’s producer statistics.

VI. EXPERIMENTAL SETUP

Datasets (labels used only for evaluation). *SynDrift*: 60k points from 15 anisotropic Gaussians in 10 dimensions, with 4 drifting, 2 emerging and 1 vanishing cluster. *NSL-KDD* [33] (60k×51, 5 classes), *Shuttle*, *Pen-Digits* and *Letter* from PMLB [34]. Three *naturally drifting real streams* kept in their original order: *Intel Lab* [35] (54 sensor motes over 5 weeks; temperature, humidity, light and voltage; *naturally partitioned* by grouping motes into 10 spatial zones; unlabeled, $k = 8$), *Gas Sensor Drift* [36] (36 months, 6 gases, 128 features), and *CoverType* [37] in the original stream order [38].

Settings. Twelve settings, each with 10 nodes and 300 s of stream: four static real datasets and SynDrift with Dirichlet(0.3) label skew [39]; the four static real datasets turned into *evolving streams* (each class gets a random birth time, as in MOA-style generators); and the three natural streams. Links alternate between WAN (80 ms, 1 % loss, 5 Mbit/s) and cellular (150 ms, 3 %, 1 Mbit/s).

Methods. *FedCAST-v3* (this paper). *Top- k magnitude*: the same budget controller and token bucket, with micro-clusters ranked by change magnitude, i.e. top- k sparsification with error feedback [9], [10]. *Periodic- Δ* : changed micro-clusters every T seconds. *k-FED* [7]: k' local centres every T seconds, with

TABLE I
FEDCAST-V3 VS. TOP- k MAGNITUDE (BOTH 8-BIT, SAME BUDGET CONTROLLER), PAIRED BY DATASET, SETTING, BUDGET AND SEED.

Setting	pairs	v3 better	mean gain	p
Static real	100	35 %	−0.003	0.97
Drifting (SynDrift)	25	80 %	+0.062	1.1×10^{-4}
Evolving real	100	68 %	+0.017	1.5×10^{-4}
Naturally drifting real	75	75 %	+0.069	2.3×10^{-6}
All non-stationary	200	72 %	+0.042	3.9×10^{-11}
All	300	60 %	+0.027	2.4×10^{-8}

the oracle k' . **All four use the same 8-bit encoding.** Budgets: $B/W \in \{2, \dots, 50\}$ B/s per node and $T \in \{30, \dots, 240\}$ s. *Centralised-Raw* streams every point to one server-side micro-cluster and is the quality reference.

Metrics and statistics. Every 10 s the coordinator’s model is evaluated on the last 60 s of data. The *cost ratio* is its k-means cost divided by that of Centralised-Raw on the same seed (lower is better). Bytes are Kafka payload plus a 24-byte record overhead, validated to within 3 % of librdkafka’s statistics. Each configuration uses 5 seeds. We report paired one-sided Wilcoxon tests (same dataset, setting, budget and seed) and Friedman tests with the Nemenyi critical difference over budget-matched blocks [40].

VII. RESULTS

A. Is Value Better Than Magnitude?

Table I is the central result. Against the strongest federated-learning compressor, under identical 8-bit encoding and identical budgets, value-based communication wins 72 % of the 200 paired comparisons on non-stationary streams ($p = 3.9 \times 10^{-11}$). It wins 75 % on the naturally drifting real streams ($p = 2.3 \times 10^{-6}$), 68 % on evolving streams and 80 % on SynDrift. On static streams the two rankings tie, with magnitude marginally ahead (mean cost-ratio difference 0.003). This is what Proposition 1 predicts: when nothing drifts, value and magnitude select nearly the same updates; under evolution they diverge.

The effect also appears without quantization. In float32 on the same seeds (250 pairs), the exact value beats magnitude in 75 % of non-stationary pairs ($p = 7.4 \times 10^{-9}$). It reduces the excess cost over the centralized learner by 86 % on SynDrift and 71 % on evolving Pen-Digits, and ties on static streams.

B. Comparison with Periodic and One-Shot Federated Clustering

Fig. 2 shows cost-bytes fronts on all twelve settings, and Fig. 3 ranks the methods over the 31 budget-matched blocks in which all four can operate (1 %, 3 % and 10 % of the Centralised-Raw bytes; Friedman $p = 6.8 \times 10^{-12}$). Budgeted sparse communication, whether value- or magnitude-ranked, dominates schedules: FedCAST-v3 and top- k are statistically tied in mean rank (1.68 and 1.61), and both are far ahead of Periodic- Δ (3.23) and k-FED (3.48). FedCAST-v3 wins the tight-budget blocks on non-stationary data. At 1 % of raw traffic it reaches 1.015 on SynDrift (top- k 1.052, k-FED 1.127)

TABLE II
DESIGN EVOLUTION: WIN RATE AGAINST TOP- k MAGNITUDE ON
NON-STATIONARY STREAMS (PAIRED; p ONE-SIDED WILCOXON).

Ranking (encoding)	pairs	win	p
v1: objective-linked upper bound (float32)	130	62 %	2.8×10^{-3}
v2: exact value, Eq. (4) (float32)	130	75 %	7.4×10^{-9}
v2: exact value (8-bit, both methods)	200	64 %	1.6×10^{-7}
v3: + tie-break + use-it-or-lose-it (8-bit)	200	72 %	3.9×10^{-11}

and 1.217 on Intel Lab (1.429, 3.284). At 3 % it reaches 1.049 on evolving Pen-Digits (1.114, 1.189) and 1.127 on Gas (1.210, 1.166). To reach a cost ratio of 1.10, it needs 18 kB on SynDrift (top- k 25, k-FED 29, Periodic- Δ 69), 23 kB on evolving Pen-Digits (32, 29, 78) and 36 kB on Intel Lab (57; neither schedule reaches 1.10). k-FED remains the most frugal method at the very lowest byte counts on some static and natural streams (Pen-Digits, Gas, CoverType), in the regime of well-separated local clusters that its theory covers.

C. How the Design Evolved (Ablation)

Table II traces the design against the same competitor. Ranking by an objective-linked upper bound (v1) helped only weakly on non-stationary data. The exact value of Proposition 1 (v2) made the advantage decisive. Under quantization, v2 plateaued at loose budgets: once all remaining changes had near-zero value it stopped sending, leaving budget unused (NSL-KDD static stayed at 1.043 at 1 %, 3 % and 10 % of raw traffic). The magnitude tie-break and use-it-or-lose-it sends of v3 fixed this and raised the win rate to 72 %. 8-bit encoding cut bytes by 2–3 $\times$ at equal quality for every method (e.g. NSL-KDD evolving reaches 1.10 with 38 kB instead of 116 kB). Among the mechanisms shared by all versions, prioritized partial deltas are essential: sending all changed micro-clusters under the same budget raised the cost ratio by up to 32 % on SynDrift. Swap search lowered the cost in 17 of 18 non-IID settings (e.g. Shuttle 1.107 against 1.218).

D. Heterogeneity and Drift

With the objective-linked policy and the same aggregator, cost ratios stayed at 1.008–1.065 from IID to one-class-per-node partitions on SynDrift, Pen-Digits and Letter, against 1.047–1.126 for periodic sending and 1.067–1.281 for k-FED. Node balancing ($N_i^{0.5}$) helped in the cluster-exclusive regime (Pen-Digits: 1.011 against 1.029). In that regime k-FED with the oracle k' reached higher label agreement (ARI 0.66) at a much higher k-means cost (1.22). On SynDrift, the value-driven trigger reported emerging clusters in 9–11 s, against 27 s for periodic updates and k-FED, while sending less (59–87 kB against 127 kB). Fig. 4 shows the time course, and Fig. 5 shows that every node stays within the hard budget envelope.

E. Systems Evaluation

Fault tolerance (real broker). We killed the coordinator with SIGKILL after 15 s and restarted it 8 s later. It restored all node states from the compacted changelog in 1.08 s, caught

TABLE III
MULTI-CONTAINER DEPLOYMENT (SINGLE RUNS; 10 EDGE CONTAINERS
WITH KERNEL BANDWIDTH CAPS; REAL BROKER; ALL METHODS 8-BIT).
UPLINK kB AND COST RATIO.

Stream	FedCAST-v3	Top- k	Periodic- Δ
SynDrift	35.9 / 1.006	40.5 / 1.019	30.1 / 1.154
Intel Lab (natural)	35.4 / 1.091	37.9 / 1.188	19.6 / 2.547

up within 0.5 s, and ended in a state *identical* to that of a shadow coordinator that never failed: the same micro-cluster identifiers and a mass difference of 0.0. All 238 summaries were applied exactly once. When we stopped the broker for 34 s in the middle of a run, all 231 summaries were delivered and applied, with no loss and no delivery errors.

Scalability. From 5 to 50 nodes on one 4-core machine (Fig. 6), end-to-end latency stayed at 33–35 ms (p50) and 57 ms (p95), against 156–179 ms (p50) for streaming raw data, which shipped 2.5–2.8 MB against FedCAST’s 32–325 kB. The complete system at 50 nodes used 788 MB of RAM.

Multi-container deployment. Each of 10 edge nodes ran in its own container, with its own network stack and real TCP, against a real broker. Kernel-enforced bandwidth caps (tc tbfb: 100 Mbit, 5 Mbit, 1 Mbit and 128 kbit, round-robin) were applied per node. Quality was evaluated offline from the coordinator’s logged models against the simulated centralized learner. Table III confirms the simulation results end to end. On the naturally partitioned Intel Lab stream, FedCAST-v3 reached 1.091 with 35 kB, against 1.188 with 38 kB for top- k and 2.547 for Periodic- Δ . Every summary was applied, median latency was 31–39 ms, and each edge container used about 110 MB.

F. What Did Not Work

We report negative results so that others need not repeat them. (i) A normalized 1:1 blend of an objective bound and magnitude looked excellent in a 3-seed pilot but was ranked *worst* in the full 250-block test. (ii) A novelty override for emerging mass was redundant with the value criterion: detection delays were identical with and without it. (iii) Link-price awareness shifted only 4–7 % of traffic away from poor links, and only under loose budgets. (iv) Compressed multi-resolution full summaries (k-FED-style coarsening driven by our trigger) were worse than both k-FED and FedCAST. (v) Data-driven adaptive resolution slid along the existing trade-off curve instead of improving it. (vi) The model-free score U of Theorem 1 gives a guarantee but ranks worse than the exact value ($p = 5 \times 10^{-4}$).

VIII. DISCUSSION AND LIMITATIONS

When to use what. For evolving streams under a byte budget, communicating by value is the right choice. For static streams, top- k magnitude with error feedback is equally good. Both should replace periodic synchronization and one-shot federated clustering, which lose on every non-stationary setting in our evaluation. For well-separated, nearly static local clusters at extremely low byte counts, k-FED remains competitive.

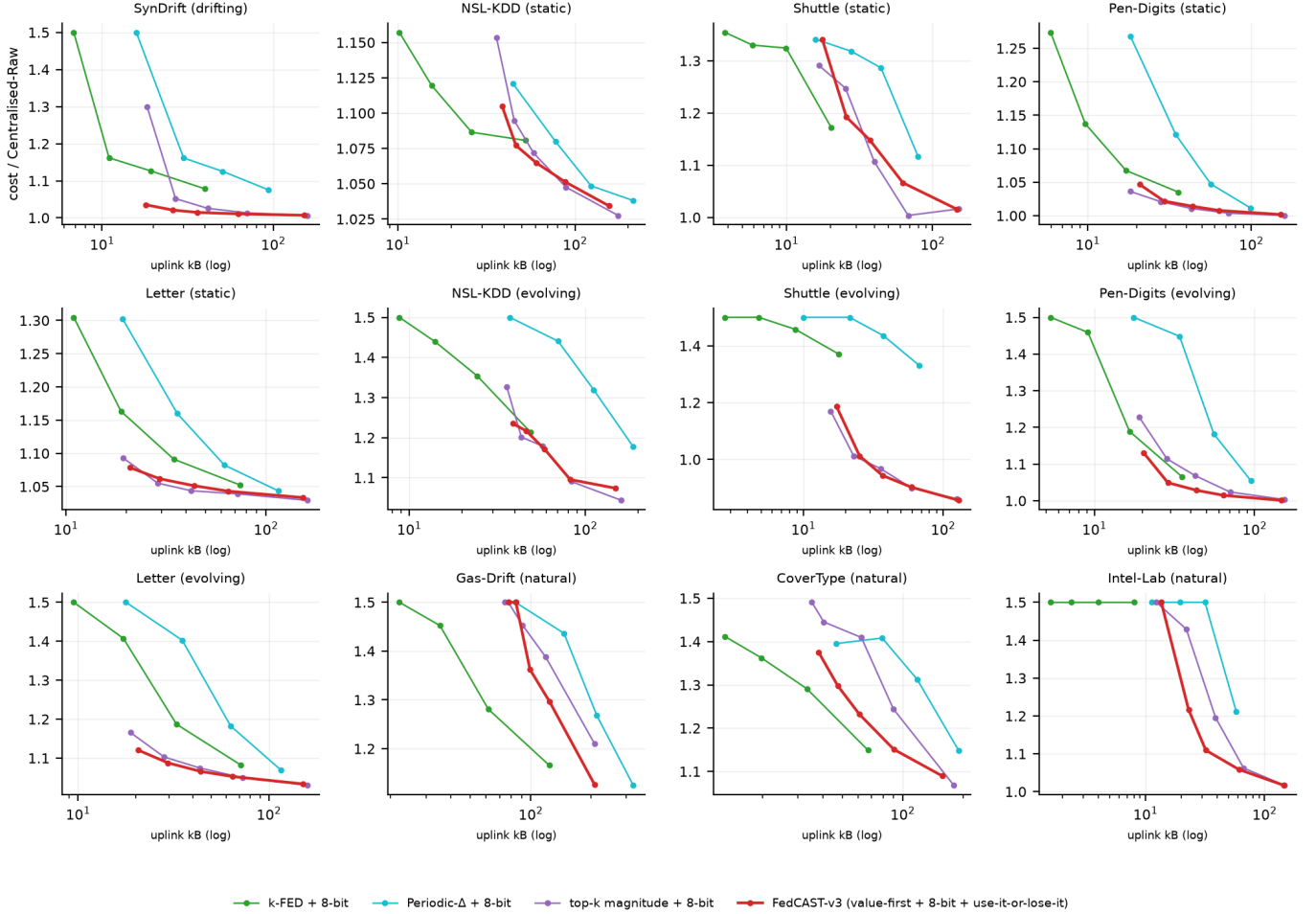


Fig. 2. Cost ratio vs. uplink bytes on twelve settings (mean of 5 seeds; all methods 8-bit). Lower-left is better. Red: FedCAST-v3; purple: top- k magnitude with the same budget controller; cyan: Periodic- Δ ; green: k-FED.

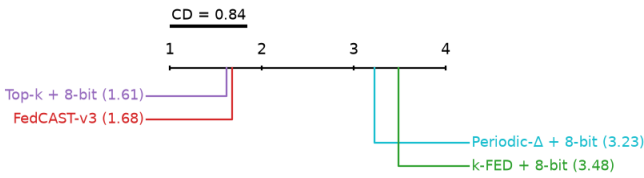


Fig. 3. Mean ranks over 31 budget-matched blocks (Nemenyi, $\alpha = 0.05$).

Kafka as part of the algorithm. Building the protocol on Kafka’s ordering, idempotence, replay and compaction removed whole classes of failure handling from the algorithm, at the cost of a broker that needs about 400 MB.

Limitations. (i) Proposition 1 assumes fixed assignments, and the marginal value ignores cross terms with other nodes’ unsent changes. (ii) The sandbox kernel lacked `netem`, so delay and loss were emulated in the application; kernel shaping covered bandwidth only, and deployment results are single runs. (iii) k is assumed known; an unknown- k aggregator [8] could replace ours without changing the send policy. (iv) CFs are aggregates but not differentially private. Gaussian noise on the quantized deltas [41] would add a bytes–accuracy–privacy trade-off that

the budget controller can also bound. (v) Streams are replayed at an accelerated rate; energy was not measured.

IX. CONCLUSION

The value of an update to a federated clustering has an exact closed form, and it is computable on the device. FedCAST uses it to decide what and when to send under a hard byte budget, on top of Apache Kafka with exactly-once state. On non-stationary streams, and most strongly on naturally drifting real data, communicating by value significantly outperforms communicating by magnitude, the federated-learning standard, as well as schedule-based federated clustering, while matching them on static streams. Future work includes an unknown- k aggregator, differential privacy for quantized deltas, a regret analysis of value-greedy sending, and deployments on physical devices over real wireless links.

REFERENCES

- [1] T. Zhang, R. Ramakrishnan, and M. Livny, “BIRCH: an efficient data clustering method for very large databases,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1996, pp. 103–114.

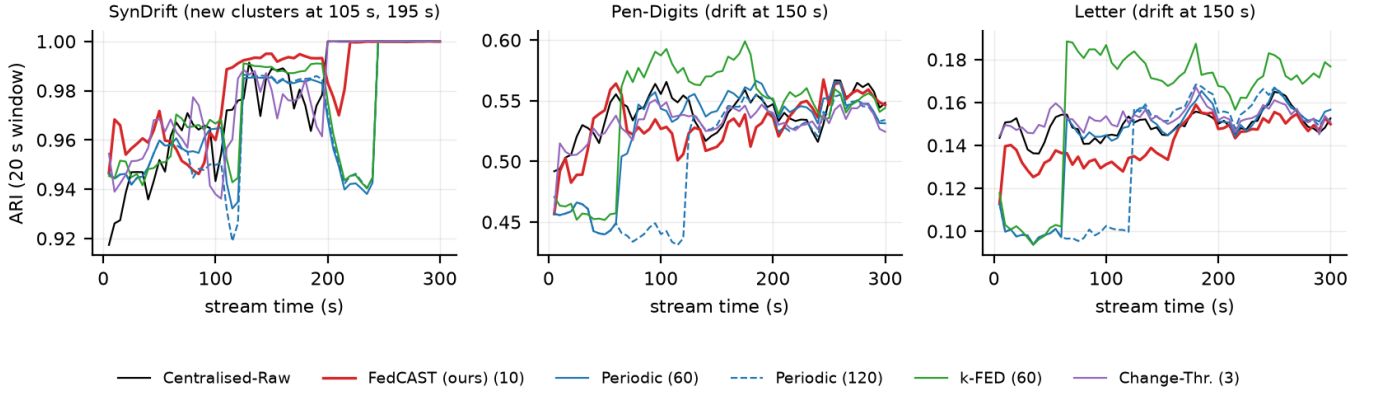


Fig. 4. ARI over time (20 s window; mean of 5 seeds). On SynDrift, new clusters emerge at 105 s and 195 s; on Pen-Digits and Letter, node class sets rotate at 150 s.

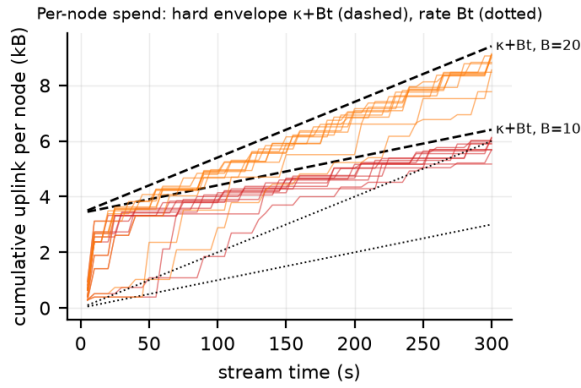


Fig. 5. Per-node cumulative uplink with $B/W = 10$ and 20 B/s. Dashed: hard envelope $\kappa + Bt$ (Proposition 2); dotted: rate Bt .

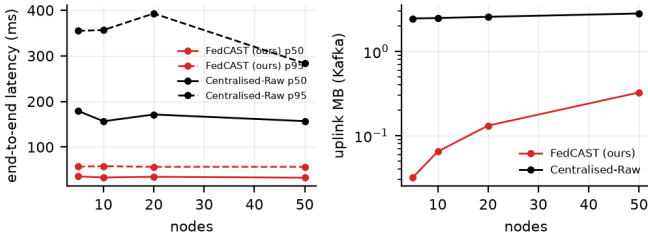


Fig. 6. Real Kafka, 5–50 nodes: end-to-end latency (left) and uplink bytes (right, log scale).

[2] C. C. Aggarwal, J. Han, J. Wang, and P. S. Yu, “A framework for clustering evolving data streams,” in *Proceedings of the 29th International Conference on Very Large Data Bases (VLDB)*, 2003, pp. 81–92.

[3] F. Cao, M. Ester, W. Qian, and A. Zhou, “Density-based clustering over an evolving data stream with noise,” in *Proceedings of the SIAM International Conference on Data Mining (SDM)*, 2006, pp. 328–339.

[4] G. Cormode, S. Muthukrishnan, and W. Zhuang, “Conquering the divide: Continuous clustering of distributed data streams,” in *Proceedings of the IEEE 23rd International Conference on Data Engineering (ICDE)*, 2007, pp. 1036–1045.

[5] J. Gama, P. P. Rodrigues, and L. Lopes, “Clustering distributed sensor data streams using local processing and reduced communication,” *Intelligent Data Analysis*, vol. 15, no. 1, pp. 3–28, 2011.

[6] D.-H. Tran and K.-U. Sattler, “Communication-efficient exact clustering of distributed streaming data,” in *Computational Science and Its*

Applications (ICCSA), ser. LNCS, vol. 7975, 2013, pp. 421–436.

[7] D. K. Dennis, T. Li, and V. Smith, “Heterogeneity for the win: One-shot federated clustering,” in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 139, 2021, pp. 2611–2620.

[8] Y. Zhang, Y. Zhang, Y. Lu, M. Li, X. Chen, and Y.-m. Cheung, “Asynchronous federated clustering with unknown number of clusters,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025.

[9] A. F. Aji and K. Heafield, “Sparse communication for distributed gradient descent,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017, pp. 440–445.

[10] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, “Sparsified SGD with memory,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 4447–4458.

[11] S. P. Karimireddy, Q. Rebjock, S. U. Stich, and M. Jaggi, “Error feedback fixes SignSGD and other gradient compression schemes,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 3252–3261.

[12] M. Hahsler and M. Bolaños, “Clustering data streams based on shared density between micro-clusters,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 6, pp. 1449–1461, 2016.

[13] A. Zubaroğlu and V. Atalay, “Data stream clustering: a review,” *Artificial Intelligence Review*, vol. 54, pp. 1201–1236, 2021.

[14] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.

[15] M.-F. Balcan, S. Ehrlich, and Y. Liang, “Distributed k-means and k-median clustering on general topologies,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013, pp. 1995–2003.

[16] M. Stallmann and A. Wilbik, “Towards federated clustering: A federated fuzzy c-means algorithm (FFCM),” *arXiv preprint arXiv:2201.07316*, 2022.

[17] M. Angelova, V. Boeva, S. Abghari, S. Ickin, and X. Lan, “FedCluLearn: Federated continual learning using stream micro-cluster indexing scheme,” in *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, 2025.

[18] P. R. Silva, J. Vinagre, and J. Gama, “Federated anomaly detection over distributed data streams,” *arXiv preprint arXiv:2205.07829*, 2022.

[19] B. Li, L. Cheng, Z. Qin, and Y. Wu, “Density-aware and cluster-based federated anomaly detection on data streams,” in *Proceedings of the 18th ACM International Conference on Web Search and Data Mining (WSDM)*, 2025.

[20] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[21] P. Kairouz, H. B. McMahan *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[22] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, “Deep gradient compression: Reducing the communication bandwidth for distributed training,” in *International Conference on Learning Representations (ICLR)*, 2018.

- [23] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, “Robust and communication-efficient federated learning from non-i.i.d. data,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 9, pp. 3400–3413, 2020.
- [24] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, “QSGD: Communication-efficient SGD via gradient quantization and encoding,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1709–1720.
- [25] S. Ghosh, B. Aquino, and V. Gupta, “EventGraD: Event-triggered communication in parallel machine learning,” *Neurocomputing*, vol. 483, pp. 474–487, 2022.
- [26] S. Zehtabi, S. Hosseinalipour, and C. G. Brinton, “Decentralized event-triggered federated learning with heterogeneous communication thresholds,” *arXiv preprint arXiv:2204.03726*, 2022.
- [27] C. Martín, P. Langendoerfer, P. S. Zarrin, M. Díaz, and B. Rubio, “Kafka-ML: Connecting the data stream with ML/AI frameworks,” *Future Generation Computer Systems*, vol. 126, pp. 15–33, 2022.
- [28] A. J. Carnerero-Cano, C. Martín, and M. Díaz, “Towards flexible data stream collaboration: Federated learning in Kafka-ML,” *Internet of Things*, vol. 25, p. 101036, 2024.
- [29] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, 2011.
- [30] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, and K. Tzoumas, “State management in Apache Flink: consistent stateful distributed stream processing,” *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1718–1729, 2017.
- [31] T. Kanungo, D. M. Mount, N. S. Netanyahu, C. D. Piatko, R. Silverman, and A. Y. Wu, “A local search approximation algorithm for k-means clustering,” *Computational Geometry*, vol. 28, no. 2–3, pp. 89–112, 2004.
- [32] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2007, pp. 1027–1035.
- [33] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009.
- [34] R. S. Olson, W. La Cava, P. Orzechowski, R. J. Urbanowicz, and J. H. Moore, “PMLB: a large benchmark suite for machine learning evaluation and comparison,” *BioData Mining*, vol. 10, no. 1, p. 36, 2017.
- [35] P. Bodik, W. Hong, C. Guestrin, S. Madden, M. Paskin, and R. Thibaux, “Intel Berkeley research lab sensor data,” <http://db.csail.mit.edu/labdata/labdata.html>, 2004.
- [36] A. Vergara, S. Vembu, T. Ayhan, M. A. Ryan, M. L. Homer, and R. Huerta, “Chemical gas sensor drift compensation using classifier ensembles,” *Sensors and Actuators B: Chemical*, vol. 166, pp. 320–329, 2012.
- [37] J. A. Blackard and D. J. Dean, “Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables,” *Computers and Electronics in Agriculture*, vol. 24, no. 3, pp. 131–151, 1999.
- [38] V. Losing, B. Hammer, and H. Wersing, “KNN classifier with self adjusting memory for heterogeneous concept drift,” in *Proceedings of the IEEE 16th International Conference on Data Mining (ICDM)*, 2016, pp. 291–300.
- [39] T.-M. H. Hsu, H. Qi, and M. Brown, “Measuring the effects of non-identical data distribution for federated visual classification,” *arXiv preprint arXiv:1909.06335*, 2019.
- [40] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [41] A. Epasto, T. Mukherjee, and P. Zhong, “Differentially private clustering in data streams,” *arXiv preprint arXiv:2307.07449*, 2023.